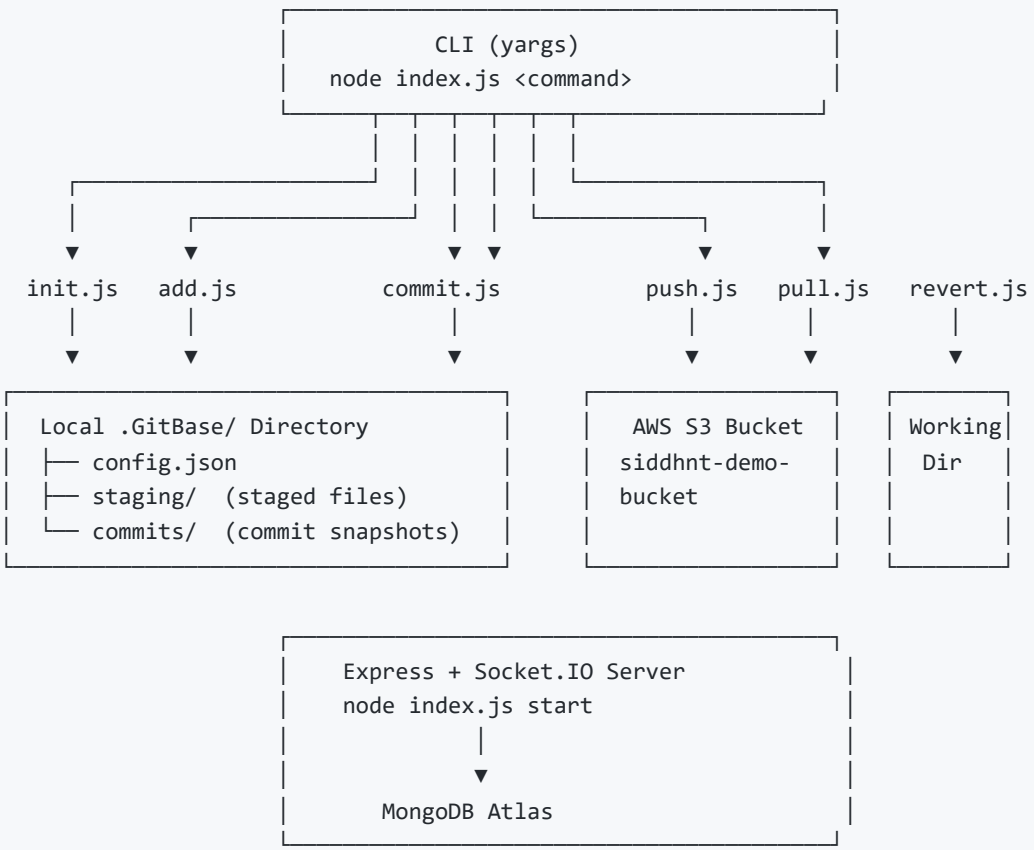
GitBase — Complete Project Code Explanation

GitBase is a **GitHub clone** — a simplified, custom-built version control system with a CLI (like `git`) and a web server (like GitHub). It stores commits locally as files and syncs them to **AWS S3** as the remote backend, with **MongoDB** for user/repo/issue data and **Socket.IO** for real-time features.

Architecture Overview



Project Folder Structure

```
GitBase/
├── .gitignore                  # Git ignore rules
└── backend/
    ├── .env                    # Environment variables (secrets)
    ├── package.json            # Project metadata & dependencies
    ├── index.js                # Entry point: CLI + web server
    ├── hello.txt               # Sample test file
    ├── commit.json             # Sample commit metadata
    ├── .GitBase/               # Local "repository" data (like .git/)
    │   ├── config.json         # Repo configuration
    │   ├── staging/            # Staged files (like git's index)
    │   │   └── hello.txt
    │   └── commits/            # Committed snapshots
    │       └── 03f5f82f-.../   # One commit (UUID)
    │           ├── commit.json # Commit metadata
    │           └── hello.txt   # Committed file snapshot
    ├── config/
    │   └── aws-config.js       # AWS S3 client configuration
    ├── controllers/
    │   ├── init.js             # `init` command handler
    │   ├── add.js              # `add` command handler
    │   ├── commit.js           # `commit` command handler
    │   ├── push.js             # `push` command handler
    │   ├── pull.js             # `pull` command handler
    │   └── revert.js           # `revert` command handler
    ├── middleware/
    │   ├── authMiddleware.js   # Authentication middleware (empty)
    │   └── authorizeMiddleware.js # Authorization middleware (empty)
    ├── models/
    │   ├── userModel.js        # User mongoose model (empty)
    │   ├── repoModel.js        # Repository mongoose model (empty)
    │   └── issueModel.js       # Issue mongoose model (empty)
    └── routes/                 # API routes (empty directory)
```

1. Root Files

.gitignore

```
# Dependencies
node_modules/

# Environment and local configuration
.env
.env.*
!.env.example
.GitBase/

# Logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Build and test output
dist/
build/
coverage/

# Operating system files
desktop.ini
Thumbs.db
```

Line-by-line:

Line	Code	Explanation
2	`node_modules/`	Ignores the entire `node_modules` directory (installed npm packages — huge, never committed)
5	`.env`	Ignores the `.env` file so secrets (DB passwords, API keys) are never pushed to Git
6	`.env.*`	Ignores all variant env files like `.env.production`, `.env.local`
7	`!.env.example`	Exception: `.env.example` (a template without real secrets) IS tracked
8	`.GitBase/`	Ignores the local GitBase repository data directory
11	`*.log`	Ignores all log files
12–14	`npm-debug.log*` etc.	Ignores specific debug/error log files from npm/yarn
17–19	`dist/`, `build/`, `coverage/`	Ignores build output and test coverage reports
22–23	`desktop.ini`, `Thumbs.db`	Ignores Windows OS auto-generated files

---

2. backend/ — The Main Application

package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "Github Clone",
  "license": "ISC",
  "author": "Siddhant",
  "type": "commonjs",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "aws-sdk": "^2.1693.0",
    "bcryptjs": "^3.0.3",
    "body-parser": "^2.4.0",
    "cors": "^2.8.6",
    "dotenv": "^18.0.9",
    "express": "^5.3.0",
    "http": "^0.0.1-security",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.11.4",
    "socket.io": "^4.8.4",
    "uuid": "^14.0.3",
    "yargs": "^18.2.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

Line-by-line:

Line	Key	Value	Explanation
2	`name`	`"backend"`	Package name for npm
3	`version`	`"1.0.0"`	Semantic version
4	`description`	`"Github Clone"`	Human-readable project description
5	`license`	`"ISC"`	Open-source ISC license
6	`author`	`"Siddhant"`	The author's name
7	`type`	`"commonjs"`	Uses `require() / module.exports` (not ES modules `import / export`)
8	`main`	`"index.js"`	Entry point file
10	`scripts.start`	`"node index.js"`	`npm start` runs `node index.js`
13	`aws-sdk`	`^2.1693.0`	AWS SDK — used to interact with S3 (upload/download commits)
14	`bcryptjs`	`^3.0.3`	Password hashing library (imported but not yet used)
15	`body-parser`	`^2.4.0`	Parses incoming JSON request bodies
16	`cors`	`^2.8.6`	Enables Cross-Origin Resource Sharing (lets frontend on different domain call this API)
17	`dotenv`	`^18.0.9`	Loads `.env` file variables into `process.env`
18	`express`	`^5.3.0`	Web framework for building the HTTP API server
19	`http`	`^0.0.1-security`	Node's HTTP module (npm shim — Node already has this built-in)
20	`jsonwebtoken`	`^9.0.3`	JWT creation/verification for authentication (imported but not yet used)
21	`mongoose`	`^9.11.4`	MongoDB ODM — models, schemas, queries
22	`socket.io`	`^4.8.4`	Real-time bidirectional event-based communication (WebSockets)
23	`uuid`	`^14.0.3`	Generates unique IDs (UUIDs) for each commit
24	`yargs`	`^18.2.0`	CLI argument parser — turns `node index.js init` into structured commands
27	`nodemon`	`^3.1.14`	Dev tool: auto-restarts server on file changes

.env

```
MONGODB_URI=mongodb+srv://siddhantsingh4work73_db_user:d6BoXlAKJiZ4F9MG@gitbase.o5trmqk.mongodb.net/
appName=GitBase
PORT=3000
```

Line	Variable	Explanation
1	`MONGODB_URI`	Full MongoDB Atlas connection string with username, password, and cluster hostname. Used by `mongoose.connect()` in `index.js`
2	`appName`	Application name identifier
3	`PORT`	The port the Express server listens on (defaults to 3000)

⚠ **CAUTION:** The `.env` file contains a real database password. This file is in `.gitignore` but was committed to the local `.GitBase` history. You should rotate this credential.

---

index.js — The Main Entry Point

This is the heart of the application. It serves **two purposes**:

1. **CLI tool** — like `git init`, `git add`, `git commit`, etc.
2. **Web server** — an Express + Socket.IO server with MongoDB

```
const express = require("express");                    // Line 1
const dotenv = require("dotenv");                      // Line 2
const cors = require("cors");                          // Line 3
const mongoose = require("mongoose");                  // Line 4
const bodyParser = require("body-parser");             // Line 5
const http = require("http");                          // Line 6

const yargs = require("yargs");                        // Line 8
const { hideBin } = require("yargs/helpers");          // Line 9
```

Lines 1–9: Imports

- `express` — creates the web server and defines routes
- `dotenv` — reads the `.env` file and puts values into `process.env`
- `cors` — middleware that adds `Access-Control-Allow-Origin` headers so browsers allow cross-origin requests
- `mongoose` — provides schema-based modeling for MongoDB documents

- `body-parser` — parses `Content-Type: application/json` request bodies into `req.body`
- `http` — Node.js built-in HTTP server; needed to attach Socket.IO to the same port as Express
- `yargs` — declarative CLI framework that parses commands like `node index.js commit "my message"`
- `hideBin` — strips the first two `process.argv` entries (`node` and the script path) so yargs only sees the user's command

```
const { initRepo } = require("./controllers/init");    // Line 11
const { addRepo } = require("./controllers/add");      // Line 12
const { commitRepo } = require("./controllers/commit");// Line 13
const { pushRepo } = require("./controllers/push");    // Line 14
const { pullRepo } = require("./controllers/pull");    // Line 15
const { revertRepo } = require("./controllers/revert");// Line 16
const { Server } = require("socket.io");               // Line 17
```

Lines 11–17: Import controller functions and Socket.IO

- Each controller handles one CLI command (`init` , `add` , `commit` , `push` , `pull` , `revert`)
- `Server` is the Socket.IO server class for real-time WebSocket connections

```
dotenv.config();                                       // Line 19
```

Line 19: Reads `.env` file and loads all key-value pairs into `process.env` . After this line, `process.env.MONGODB_URI` , `process.env.PORT` , etc. are all available.

```
yargs(hideBin(process.argv))                           // Line 21
  .command("start", "Start a new server", {}, startServer)  // Line 22
  .command("init", "Initialize a new repository", {}, initRepo) // Line 23
  .command(                                            // Line 24
    "add <file>",                                      // Line 25
    "Add a file to the repository",                    // Line 26
    (yargs) => {                                       // Line 27
      yargs.positional("file", {                       // Line 28
        describe: " File to add to the staging area",  // Line 29
        type: "string",                                // Line 30
      });                                              // Line 31
    },                                                 // Line 32
    (argv) => {                                        // Line 33
      addRepo(argv.file);                              // Line 34
    },                                                 // Line 35
  )                                                    // Line 36
```

Lines 21–36: CLI Command Definitions

`yargs(hideBin(process.argv))` creates a CLI parser. Each `.command()` call registers a command:

Command	Usage	What it does
`start`	`node index.js start`	Starts the Express web server
`init`	`node index.js init`	Creates the `.gitBase` directory structure
`add <file>`	`node index.js add hello.txt`	Copies a file to the staging area
`commit <msg>`	`node index.js commit "First commit"`	Creates a commit snapshot from staged files
`push`	`node index.js push`	Uploads all commits to AWS S3
`pull`	`node index.js pull`	Downloads all commits from AWS S3
`revert <commitID>`	`node index.js revert abc-123`	Restores files from a specific commit

The `<file>` and `<msg>` in angle brackets mean they are **required positional parameters**. The `builder` function (e.g., lines 27–32) defines the parameter metadata. The `handler` function (e.g., lines 33–35) is the callback that runs when the command is invoked.

```
.command(                                          // Line 37
  "commit <message>",                              // Line 38
  "Commit the staged file",                        // Line 39
  (yargs) => {                                     // Line 40
    yargs.positional("message", {                  // Line 41
      describe: " Commit message",                 // Line 42
      type: "string",                              // Line 43
    });                                            // Line 44
  },                                               // Line 45
  (argv) => {                                      // Line 46
    commitRepo(argv.message);                      // Line 47
  },                                               // Line 48
)                                                  // Line 49
.command("push", "Push commits to S3", {}, pushRepo) // Line 50
.command("pull", "Pull commits to S3", {}, pullRepo) // Line 51
.command(                                          // Line 52
  "revert <commitID>",                             // Line 53
  "Revert to a specific commit",                   // Line 54
  (yargs) => {                                     // Line 55
    yargs.positional("commitID", {                 // Line 56
      describe: "Commit ID to revert to",          // Line 57
      type: "string",                              // Line 58
    });                                            // Line 59
  },                                               // Line 60
  (argv) => {                                      // Line 61
    revertRepo(argv.commitID);                     // Line 62
  },                                               // Line 63
)                                                  // Line 64

.demandCommand(1, "You need atleast one command")  // Line 66
.help().argv;                                      // Line 67
```

- **Line 66:** `.demandCommand(1, ...)` — forces the user to provide at least one command; shows the error message if they just run `node index.js` with nothing
- **Line 67:** `.help()` adds `--help` flag support. `.argv` triggers yargs to actually parse `process.argv` and execute the matched command

```
function startServer() {                              // Line 69
  const app = express();                              // Line 70
  const port = process.env.PORT || 3000;              // Line 71

  app.use(bodyParser.json());                         // Line 73
  app.use(express.json());                            // Line 74
```

Lines 69–74: Create Express App

- **Line 70:** Creates a new Express application instance
- **Line 71:** Reads port from environment variable, defaults to 3000 if not set
- **Line 73:** `bodyParser.json()` middleware — parses incoming JSON request bodies, making the data available as `req.body`
- **Line 74:** `express.json()` — Express's built-in JSON parser (does the same as line 73 — redundant, either one would suffice)

```
  const mongoURI = process.env.MONGODB_URI;           // Line 76
  mongoose                                            // Line 77
    .connect(mongoURI)                                // Line 78
    .then(() => console.log("MongoDB connected!"))    // Line 79
    .catch((err) => console.error("Unable to connect to MongoDB: ", err)); // Line 80
```

Lines 76–80: Connect to MongoDB

- **Line 76:** Gets the MongoDB connection string from `.env`
- **Lines 77–80:** `mongoose.connect()` returns a Promise. On success — logs "MongoDB connected!". On failure — logs the error.

```
  app.use(cors({ origin: "*" }));                     // Line 82

  app.get("/", (req, res) => {                        // Line 84
    res.send("Welcome!");                             // Line 85
  });                                                 // Line 86

  let user = "test";                                  // Line 88
```

- **Line 82:** Enables CORS for **all origins** (`*`). Any website can call this API.
- **Lines 84–86:** Defines a GET route at `/` that responds with "Welcome!" — a simple health check
- **Line 88:** Initializes a `user` variable (gets updated when a WebSocket client joins)

```
const httpServer = http.createServer(app);           // Line 90
const io = new Server(httpServer, {                  // Line 91
  cors: {                                            // Line 92
    origin: "*",                                     // Line 93
    methods: ["GET", "POST"],                        // Line 94
  },                                                 // Line 95
});                                                  // Line 96
```

Lines 90–96: Create HTTP + Socket.IO Server

- **Line 90:** Wraps the Express `app` in a raw `http.Server`. This is needed so both Express routes AND Socket.IO can share the same port.
- **Lines 91–96:** Creates a Socket.IO server attached to `httpServer`. CORS is configured to allow any origin with GET/POST methods.

```
io.on("connection", (socket) => {                    // Line 98
  socket.on("joinRoom", (userID) => {                // Line 99
    user = userID;                                   // Line 100
    console.log("=====");                            // Line 101
    console.log(user);                               // Line 102
    console.log("=====");                            // Line 103
    socket.join(userID);                             // Line 104
  });                                                // Line 105
});                                                  // Line 106
```

Lines 98–106: Socket.IO Event Handling

- **Line 98:** Fires when any client connects via WebSocket
- **Line 99:** Listens for a custom `"joinRoom"` event from the connected client
- **Line 100:** Stores the user ID (note: this overwrites for all connections — a bug for multi-user)
- **Lines 101–103:** Debug logging
- **Line 104:** `socket.join(userID)` — adds this socket to a "room" named by the user's ID. Rooms enable targeted message broadcasting (e.g., sending notifications only to that user)

```
const db = mongoose.connection;                      // Line 108

db.once("open", async () => {                        // Line 110
  console.log("CRUD operations called");             // Line 111
});                                                  // Line 112

httpServer.listen(port, () => {                      // Line 114
  console.log(`Server is running on PORT ${port}`);  // Line 115
});                                                  // Line 116
}                                                    // Line 117
```

- **Lines 108–112:** `db.once("open", ...)` fires once when the MongoDB connection opens successfully. Currently just logs — this is a hook where you'd set up Change Streams or initial DB operations.

- **Lines 114–116:** Starts the HTTP server (Express + Socket.IO) listening on the configured port. Logs a confirmation message.

---

3. config/ — Configuration

aws-config.js

```
const AWS = require("aws-sdk");                        // Line 1

AWS.config.update({ region: "ap-south-1"});            // Line 3

const s3 = new AWS.S3();                               // Line 5
const S3_BUCKET = "siddhnt-demo-bucket";               // Line 6

module.exports = {s3, S3_BUCKET};                      // Line 8
```

Line	Code	Explanation
1	`require("aws-sdk")`	Imports the AWS SDK for JavaScript
3	`AWS.config.update(...)`	Sets the AWS region to `ap-south-1` (Mumbai). AWS credentials are expected from environment variables (`AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY`) or from `~/.aws/credentials`
5	`new AWS.S3()`	Creates an S3 client instance — used to upload, download, and list objects
6	`S3_BUCKET = "siddhnt-demo-bucket"`	The S3 bucket name where commits are stored remotely
8	`module.exports`	Exports both `s3` (the client) and `S3_BUCKET` (the bucket name) for use in `push.js` and `pull.js`

---

4. controllers/ — CLI Command Handlers

Each controller implements one "git-like" command. These are the core version control operations.

init.js — `node index.js init`

Purpose: Initializes a new GitBase repository (equivalent to `git init`)

```
const fs = require("fs").promises;                    // Line 1
const path = require("path");                         // Line 2
```

- **Line 1:** Imports the **Promise-based** filesystem API. All methods (`mkdir` , `writeFile` , etc.) return Promises, enabling `async` / `await`
- **Line 2:** Imports `path` for cross-platform file path manipulation

```
async function initRepo() {                           // Line 4
  const repoPath = path.resolve(process.cwd(), ".gitBase"); // Line 5
  const commitsPath = path.join(repoPath, "commits");  // Line 6
```

- **Line 4:** `async` function — can use `await` inside
- **Line 5:** `path.resolve(process.cwd(), ".gitBase")` — resolves to an absolute path: the `.gitBase` directory in the current working directory (wherever the user runs the command)
- **Line 6:** `path.join(repoPath, "commits")` — creates path like `/project/.gitBase/commits`

```
  try {                                               // Line 8
    await fs.mkdir(repoPath, { recursive: true });    // Line 9
    await fs.mkdir(commitsPath, { recursive: true }); // Line 10
    await fs.writeFile(                               // Line 11
      path.join(repoPath, "config.json"),             // Line 12
      JSON.stringify({ bucket: process.env.S3_BUCKET }),// Line 13
    );                                                // Line 14
    console.log("Repository initialized!")            // Line 15
  } catch (err) {                                     // Line 16
    console.error("Error initialising repository", err);// Line 17
  }                                                   // Line 18
}                                                     // Line 19
```

- **Line 9:** Creates `.gitBase/` directory. `{ recursive: true }` means it won't error if it already exists, and creates parent directories as needed
- **Line 10:** Creates `.gitBase/commits/` directory
- **Lines 11–14:** Creates `config.json` inside `.gitBase/` containing the S3 bucket name as JSON
- **Line 15:** Success message
- **Lines 16–18:** If anything fails, catch and log the error

```
module.exports = { initRepo };                        // Line 21
```

- **Line 21:** Exports the function so `index.js` can import and register it as a CLI command

---

add.js — `node index.js add <file>`

Purpose: Stages a file for the next commit (equivalent to `git add`)

```
const fs = require("fs").promises;                    // Line 1
const path = require("path");                         // Line 2

async function addRepo(filePath) {                    // Line 4
  const repoPath = path.resolve(process.cwd(), ".GitBase"); // Line 5
  const stagingPath = path.join(repoPath, "staging");  // Line 6
```

- **Line 4:** Receives `filePath` — the file the user wants to stage (e.g., `"hello.txt"`)
- **Line 5:** Locates the `.GitBase` repository in the current directory
- **Line 6:** The staging area path: `.GitBase/staging/`

```
  try {                                               // Line 8
    await fs.mkdir(stagingPath, { recursive: true });  // Line 9
    const fileName = path.basename(filePath);          // Line 10
    await fs.copyFile(filePath, path.join(stagingPath, fileName)); // Line 11
    console.log(`File ${fileName} added to the staging area!`); // Line 12
  } catch (err) {                                     // Line 13
    console.error("Error adding file: ", err);        // Line 14
  }                                                   // Line 15
}                                                     // Line 16

module.exports = { addRepo };                         // Line 18
```

- **Line 9:** Creates the `staging/` directory if it doesn't exist
- **Line 10:** `path.basename(filePath)` — extracts just the filename from a path (e.g., `"/home/user/hello.txt"` → `"hello.txt"`)
- **Line 11: Copies** the file from its original location into the staging area. This is the key operation — it snapshots the file at this moment in time
- **Line 12:** Confirmation message

IMPORTANT: The staging area acts exactly like Git's index/staging area. Files are copied here first, and only staged files get included in the next commit.

---

commit.js — `node index.js commit <message>`

Purpose: Creates a snapshot of all staged files (equivalent to `git commit`)

```
const fs = require("fs").promises;                    // Line 1
const path = require("path");                         // Line 2
const { v4: uuidv4 } = require("uuid");              // Line 3
```

- **Line 3:** Imports `uuidv4` — generates a unique identifier for each commit (like Git's SHA hash, but random UUID instead)

```
async function commitRepo(message) {                  // Line 5
  const repoPath = path.resolve(process.cwd(), ".GitBase"); // Line 6
  const stagedPath = path.join(repoPath, "staging");  // Line 7
  const commitPath = path.join(repoPath, "commits");  // Line 8
```

- **Line 5:** Receives the commit `message` from the CLI
- **Line 7:** Path to staged files
- **Line 8:** Path to commits directory

```
  try {                                               // Line 10
    const commitID = uuidv4();                        // Line 11
    const commitDir = path.join(commitPath, commitID); // Line 12
    await fs.mkdir(commitDir, { recursive: true });   // Line 13
```

- **Line 11:** Generates a UUID like `"03f5f82f-3128-4f98-8ed6-75f4a5caf8bd"` — this uniquely identifies this commit
- **Line 12:** Each commit gets its own directory: `.GitBase/commits/03f5f82f-.../`
- **Line 13:** Creates that commit directory

```
    const files = await fs.readdir(stagedPath);       // Line 15
    for (const file of files) {                       // Line 16
      await fs.copyFile(                              // Line 17
        path.join(stagedPath, file),                  // Line 18
        path.join(commitDir, file),                   // Line 19
      );                                              // Line 20
    }                                                 // Line 21
```

- **Line 15:** Reads all filenames from the staging area
- **Lines 16–21:** Loops through each staged file and copies it into the new commit directory. This creates a **full snapshot** of all staged files (not a diff — the entire file is stored per commit)

```
    await fs.writeFile(                                  // Line 23
      path.join(commitDir, "commit.json"),               // Line 24
      JSON.stringify({ message, date: new Date().toISOString() }), // Line 25
    );                                                   // Line 26

    console.log(`Commit ${commitID} created with message: ${message}`); // Line 28
  } catch (err) {                                        // Line 29
    console.error("Error commiting files: ", err);       // Line 30
  }                                                      // Line 31
}                                                        // Line 32

module.exports = { commitRepo };                         // Line 34
```

- **Lines 23–26:** Creates `commit.json` inside the commit directory containing:
  - `message` — the user's commit message (e.g., `"First commit"`)
  - `date` — ISO 8601 timestamp (e.g., `"2026-10-30T18:59:01.798Z"`)
- **Line 28:** Logs the commit ID and message

---

push.js — `node index.js push`

Purpose: Uploads all local commits to AWS S3 (equivalent to `git push`)

```
const fs = require("fs").promises;                       // Line 1
const path = require("path");                            // Line 2
const { s3, S3_BUCKET } = require("../config/aws-config"); // Line 3
```

- **Line 3:** Imports the configured S3 client and bucket name

```
async function pushRepo() {                              // Line 5
  const repoPath = path.resolve(process.cwd(), ".GitBase"); // Line 6
  const commitsPath = path.join(repoPath, "commits");    // Line 7

  try {                                                  // Line 9
    const commitDirs = await fs.readdir(commitsPath);    // Line 10
    for (const commitDir of commitDirs) {                // Line 11
      const commitPath = path.join(commitsPath, commitDir); // Line 12
      const files = await fs.readdir(commitPath);        // Line 13
```

- **Line 10:** Lists all commit directories (each is a UUID)
- **Lines 11–13:** For each commit, lists all files inside it

```
      for (const file of files) {                      // Line 15
        const filePath = path.join(commitPath, file);  // Line 16
        const fileContent = await fs.readFile(filePath);// Line 17
        const params = {                               // Line 18
          Bucket: S3_BUCKET,                           // Line 19
          Key: `commits/${commitDir}/${file}`,         // Line 20
          Body: fileContent,                           // Line 21
        };                                             // Line 22

        await s3.upload(params).promise();             // Line 24
      }                                                // Line 25
    }                                                  // Line 26

    console.log("All commits pushed to S3!");          // Line 28
  } catch (err) {                                      // Line 30
    console.error("Error pushing to S3: ", err);       // Line 31
  }                                                    // Line 32
}                                                      // Line 33

module.exports = { pushRepo };                         // Line 35
```

- **Line 17:** Reads the file content as a Buffer
- **Lines 18–22:** Constructs the S3 upload parameters:
  - `Bucket` — target S3 bucket
  - `Key` — the object path in S3: `commits/<commitID>/<filename>` (mirrors the local directory structure)
  - `Body` — the file content
- **Line 24:** `s3.upload(params).promise()` — uploads the file to S3 and returns a Promise (we `await` it)
- **Line 28:** After all files in all commits are uploaded, logs success

---

pull.js — `node index.js pull`

Purpose: Downloads all commits from AWS S3 to local (equivalent to `git pull`)

```
const fs = require("fs").promises;                    // Line 1
const path = require("path");                         // Line 2
const { s3, S3_BUCKET } = require("../config/aws-config"); // Line 3

async function pullRepo() {                           // Line 5
  const repoPath = path.resolve(process.cwd(), ".GitBase"); // Line 6
  const commitPath = path.join(repoPath, "commits");  // Line 7

  try {                                               // Line 9
    const data = await s3                             // Line 10
      .listObjectsV2({                                // Line 11
        Bucket: S3_BUCKET,                            // Line 12
        Prefix: "commits/",                           // Line 13
      })                                              // Line 14
      .promise();                                     // Line 15
```

- **Lines 10–15:** `s3.listObjectsV2()` lists all objects in the S3 bucket whose key starts with `"commits/"`. This returns metadata about all commit files in the cloud. `.promise()` converts the callback-based API to a Promise.

```
    const objects = data.Contents;                    // Line 17

    for (const object of objects) {                   // Line 19
      const key = object.Key;                         // Line 20
      const commitDir = path.join(                    // Line 21
        commitPath,                                   // Line 22
        path.dirname(key).split("/").pop(),           // Line 23
      );                                              // Line 24
```

- **Line 17:** `data.Contents` is an array of S3 objects, each with a `Key` (e.g., `"commits/abc-123/hello.txt"`)
- **Line 20:** Gets the full S3 key
- **Lines 21–24:** Extracts the commit UUID from the key:
  - `path.dirname("commits/abc-123/hello.txt")` → `"commits/abc-123"`
  - `.split("/").pop()` → `"abc-123"`
  - Joins with `commitPath` to get the local directory for this commit

```
      await fs.mkdir(commitDir, { recursive: true });  // Line 26

      const params = {                                 // Line 28
        Bucket: S3_BUCKET,                             // Line 29
        Key: key,                                      // Line 30
      };                                               // Line 31

      const fileContent = await s3.getObject(params).promise(); // Line 33
      await fs.writeFile(path.join(repoPath, key), fileContent.Body); // Line 34

      console.log("All commits pulled from S3!");      // Line 36
    }                                                  // Line 37
  } catch (err) {                                      // Line 38
    console.error("Unable to pull: ", err);            // Line 39
  }                                                    // Line 40
}                                                      // Line 41

module.exports = { pullRepo };                         // Line 43
```

- **Line 26:** Creates the local commit directory
- **Line 33:** `s3.getObject()` downloads the file content from S3
- **Line 34:** Writes the downloaded content to the local filesystem, preserving the same path structure

NOTE: The "All commits pulled from S3!" log (line 36) is inside the loop, so it prints once per file. It should ideally be outside the loop.

---

revert.js — `node index.js revert <commitID>`

Purpose: Restores files from a specific commit back to the working directory (equivalent to `git checkout <commit>`)

```
const fs = require("fs");                              // Line 1
const path = require("path");                          // Line 2
const {promisify} = require("util");                   // Line 3

const readdir = promisify(fs.readdir);                 // Line 5
const copyFile = promisify(fs.copyFile);               // Line 6
```

- **Line 1:** Imports the **callback-based** `fs` module (not the `.promises` API used in other files)
- **Line 3:** `promisify` converts callback-based functions into Promise-returning functions
- **Lines 5–6:** Manually promisifies `readdir` and `copyFile`. This achieves the same result as `require("fs").promises` but is the older pattern

```
async function revertRepo(commitID) {                  // Line 8
  const repoPath = path.resolve(process.cwd(), ".GitBase"); // Line 9
  const commitsPath = path.join(repoPath, "commits");  // Line 10

  try{                                                 // Line 12
    const commitDir = path.join(commitsPath, commitID);// Line 13
    const files = await readdir(commitDir);            // Line 14
    const parentDir = path.resolve(repoPath, "..");    // Line 15

    for(const file of files){                          // Line 17
      await copyFile(path.join(commitDir, file), path.join(parentDir, file)); // Line 18
    }                                                  // Line 19

    console.log(`Commit ${commitID} reverted successfully!`); // Line 21
  } catch(err){                                        // Line 22
    console.error("Unable to revert: ", err);          // Line 23
  }                                                    // Line 24
}                                                      // Line 25

module.exports = { revertRepo };                       // Line 27
```

- **Line 13:** Constructs the path to the target commit directory
- **Line 14:** Lists all files in that commit
- **Line 15:** `path.resolve(repoPath, "..")` — gets the **parent** of `.GitBase/`, which is the project's root directory (the working directory)
- **Lines 17–19:** Copies every file from the commit directory back to the project root, effectively restoring the state of the working directory to what it was at that commit
- **Line 21:** Success message

---

5. middleware/ — Express Middleware (Empty / Placeholder)

authMiddleware.js

Currently empty. Intended for **authentication** middleware — would verify that incoming API requests include a valid JWT token. Typical implementation would:

1. Extract `Authorization: Bearer <token>` from the request header
2. Verify the JWT using `jsonwebtoken`
3. Attach the decoded user info to `req.user`
4. Call `next()` to proceed, or return 401

authorizeMiddleware.js

Currently empty. Intended for **authorization** middleware — would check whether the authenticated user has the right **permissions** (e.g., can this user push to this repository?).

---

6. models/ — Mongoose Database Models (Empty / Placeholder)

userModel.js

Currently empty. Would define a Mongoose schema for users (username, email, hashed password, etc.).

repoModel.js

Currently empty. Would define a Mongoose schema for repositories (name, owner, description, visibility, etc.).

issueModel.js

Currently empty. Would define a Mongoose schema for issues (title, body, status, assignee, etc.) — like GitHub Issues.

---

7. routes/ — API Route Handlers (Empty Directory)

Currently empty. Would contain Express router files like:

- `userRoutes.js` — signup, login, profile endpoints
- `repoRoutes.js` — create repo, list repos, etc.
- `issueRoutes.js` — create issue, comment, close, etc.

---

8. .GitBase/ — Local Repository Data

This is the **data directory** created by the `init` command. It's the equivalent of Git's `.git/` directory.

config.json

```
{}
```

Contains repository configuration. Currently empty (the `init` command writes the S3 bucket name here, but `process.env.S3_BUCKET` was undefined when it ran).

staging/ — The Staging Area

staging/hello.txt

```
hello!
```

A staged file — copied here by running `node index.js add hello.txt`.

commits/ — Commit History

commits/03f5f82f-.../commit.json

```
{"message":"First commit","date":"2026-10-30T18:59:01.798Z"}
```

Metadata for the first commit — the message and the timestamp.

commits/03f5f82f-.../hello.txt

```
hello!
```

The snapshot of `hello.txt` at the time of this commit.

Complete Data Flow

```
User runs command              What happens internally
─────────────────              ──────────────────────────

node index.js init       →    Create .gitBase/ and .gitBase/commits/

node index.js add        →    Copy hello.txt → .GitBase/staging/hello.txt
     hello.txt

node index.js commit     →    1. Generate UUID
     "First commit"           2. Copy staging/* → .GitBase/commits/<uuid>/
                               3. Write commit.json (message + date)

node index.js push       →    Read all commits → Upload each file to
                               s3://bucket/commits/<uuid>/<file>

node index.js pull       →    List S3 objects with prefix "commits/" →
                               Download each → Write to .GitBase/commits/

node index.js revert     →    Copy .GitBase/commits/<uuid>/* →
     <uuid>                    project root directory
```

Summary Table

Folder / File	Purpose	Status
`index.js`	CLI parser + web server entry point	✅ Working
`config/aws-config.js`	AWS S3 client setup	✅ Working
`controllers/init.js`	Initialize local repository	✅ Working
`controllers/add.js`	Stage files	✅ Working
`controllers/commit.js`	Create commit snapshots	✅ Working
`controllers/push.js`	Upload commits to S3	✅ Working
`controllers/pull.js`	Download commits from S3	✅ Working
`controllers/revert.js`	Restore files from a commit	✅ Working
`middleware/authMiddleware.js`	JWT authentication	⬜ Empty
`middleware/authorizeMiddleware.js`	Permission checking	⬜ Empty
`models/userModel.js`	User database schema	⬜ Empty
`models/repoModel.js`	Repository database schema	⬜ Empty
`models/issueModel.js`	Issue database schema	⬜ Empty
`routes/`	API route definitions	⬜ Empty

Part 2 — Deep Dive

How Each Dependency Works Under the Hood

express (v5.3.0)

Express is the web framework that powers the HTTP server in `startServer()`.

```
const app = express();   // Creates an Express "application"
```

The call chain in `index.js`:

```
express()  →  app.use(bodyParser.json())  →  app.use(cors(...))  →  app.get("/", ...)  →  httpServer.
```

Express works as a **middleware pipeline**. Every incoming HTTP request flows through each `app.use()` in order:

```
Incoming HTTP Request → bodyParser.json() → cors() → Route Handler → Response sent
```

- `app.use(middleware)` — registers middleware that runs on **every** request
- `app.get(path, handler)` — registers a handler only for GET requests to that path
- `req` — the request object (headers, body, params, query)
- `res` — the response object (`.send()`, `.json()`, `.status()`)

---

body-parser (v2.4.0) vs express.json()

In index.js, both are used:

```
app.use(bodyParser.json());   // Line 73 — from the body-parser package
app.use(express.json());      // Line 74 — built into Express
```

These do the exact same thing. Both parse incoming requests with `Content-Type: application/json` and populate `req.body`. In Express 4.16+, `express.json()` is actually just `body-parser` re-exported. **Only one is needed** — line 73 or 74 can be removed.

What the middleware does internally:

1. Checks if the request has `Content-Type: application/json`
2. Reads the raw request body stream (binary data)
3. Converts it to a UTF-8 string
4. Calls `JSON.parse()` on it
5. Assigns the result to `req.body`

Without this middleware, `req.body` would be `undefined`.

---

cors (v2.8.6)

```
app.use(cors({ origin: "*" }));   // Line 82
```

Problem CORS solves: Browsers block requests from one origin (e.g., `http://localhost:5173`) to a different origin (e.g., `http://localhost:3000`) for security. This is called the **Same-Origin Policy**.

What `cors()` does: Adds these HTTP response headers:

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
Access-Control-Allow-Headers: Content-Type
```

The `*` means **any website** can call this API. In production, you'd restrict it:

```
app.use(cors({ origin: "https://gitbase.com" }));  // Only allow your frontend
```

---

dotenv (v18.0.9)

```
dotenv.config();   // Line 19
```

This single line:

1. Looks for a file named `.env` in the current working directory
2. Reads it line by line
3. Parses each `KEY=VALUE` pair
4. Sets `process.env.KEY = "VALUE"` for each one

After this call:

- `process.env.MONGODB_URI` → `"mongodb+srv://siddhantsingh4work..."`
- `process.env.PORT` → `"3000"` (always a string)
- `process.env.appName` → `"GitBase"`

IMPORTANT: `process.env` values are always **strings**. Even though `PORT=3000` looks like a number, `process.env.PORT` is the string `"3000"`. JavaScript auto-coerces it in `|| 3000` so it works, but be aware.

mongoose (v9.11.4)

Mongoose is an **Object Document Mapper (ODM)** for MongoDB. It lets you define schemas and interact with MongoDB using JavaScript objects instead of raw queries.

```
mongoose.connect(mongoURI)              // Connects to MongoDB Atlas cluster
  .then(() => console.log("MongoDB connected!"))
  .catch((err) => console.error(...));
```

The connection string anatomy:

```
mongodb+srv://USERNAME:PASSWORD@CLUSTER_HOST/DATABASE_NAME
             |                 |             |
             siddhantsingh...  gitbase.o5t... (empty = default db)
```

- `mongodb+srv://` — protocol (SRV means DNS-based discovery of replica set members)
- Username/password authenticate to the cluster
- The cluster hostname points to MongoDB Atlas (cloud-hosted)
- Database name is omitted, so it uses the default `test` database

---

socket.io (v4.8.4)

Socket.IO enables **real-time, bidirectional** communication between client and server using WebSockets (with HTTP long-polling as fallback).

```
const httpServer = http.createServer(app);     // Wrap Express in raw HTTP server
const io = new Server(httpServer, { ... });    // Attach Socket.IO to the same server
```

Why is `http.createServer(app)` needed?

- Express's `app.listen()` internally calls `http.createServer(app).listen()`
- But Socket.IO needs a reference to the raw `http.Server` to "hijack" the HTTP upgrade mechanism for WebSockets
- So we create it explicitly and pass it to both Express and Socket.IO

The Socket.IO event flow in this project:

```
Frontend Client                Socket.IO Server
──────────────                 ────────────────
     │                              │
     │──── connect ─────────────────→│  (automatic on page load)
     │←──── connection acknowledged ─│
     │                              │
     │──── emit("joinRoom",  ──────→│  socket.join("user123")
     │     "user123")               │  User is now in room "user123"
     │                              │
     │                              │  Later, server can broadcast:
     │←──── io.to("user123")  ──────│  .emit("notification", data)
     │     .emit(...)               │
```

Rooms are a Socket.IO concept — a channel that sockets can join/leave. They enable **targeted broadcasting**: instead of sending a message to all connected clients, you can send only to clients in a specific room.

---

yargs (v18.2.0)

Yargs is a CLI framework. It turns raw command-line arguments into structured, validated commands.

When you run:

```
node index.js commit "First commit"
```

`process.argv` is:

```
["C:\\...\\node.exe", "S:\\GitBase\\backend\\index.js", "commit", "First commit"]
```

`hideBin(process.argv)` strips the first two entries:

```
["commit", "First commit"]
```

Then yargs matches this against the registered commands:

```
.command("commit <message>", "Commit the staged file", builderFn, handlerFn)
```

It sees `"commit"` matches the command name, `"First commit"` fills the `<message>` positional parameter, and calls `handlerFn({ message: "First commit" })`.

The `.demandCommand(1, "...")` means if the user runs `node index.js` with no command at all, yargs prints:

```
You need atleast one command
```

The `.help()` adds a `--help` flag. Running `node index.js --help` prints auto-generated usage documentation.

---

uuid (v14.0.3)

```
const { v4: uuidv4 } = require("uuid");
const commitID = uuidv4();   // → "03f5f82f-3128-4f98-8ed6-75f4a5caf8bd"
```

- `v4` means UUID version 4 — **randomly generated** (122 random bits)
- The destructuring `{ v4: uuidv4 }` imports the `v4` function and renames it to `uuidv4`
- Each call produces a unique 36-character string in the format `xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx`
- The probability of collision is astronomically low

This is used as the **commit ID** (like Git's SHA-1 hash, but random rather than content-derived).

---

bcryptjs (v3.0.3) — Imported but not yet used

Listed in `package.json` but not `require()`d anywhere. It's intended for the authentication system:

```
// How it would be used in userModel.js or auth routes:
const bcrypt = require("bcryptjs");

const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash("myPassword123", salt);
// hashedPassword → "$2a$10$N9qo8uLOickgx2ZMRZoMy..."

const isMatch = await bcrypt.compare("myPassword123", hashedPassword);  // true
```

Bcrypt is a **one-way hashing algorithm** designed for passwords. You never store the raw password — you store the hash and compare against it.

---

jsonwebtoken (v9.0.3) — Imported but not yet used

Listed in `package.json` but not `require()`d anywhere. It's intended for the auth middleware:

```
// How it would be used:
const jwt = require("jsonwebtoken");

// Create a token (during login):
const token = jwt.sign({ userId: "abc123", role: "admin" }, "SECRET_KEY", { expiresIn: "1h" });
// token → "eyJhbGciOiJIUzI1NiIs..."

// Verify a token (in middleware):
const decoded = jwt.verify(token, "SECRET_KEY");
// decoded → { userId: "abc123", role: "admin", iat: ..., exp: ... }
```

JWT (JSON Web Token) is a stateless authentication mechanism — the server doesn't need to store sessions.

GitBase vs Real Git — How the Version Control Model Works

Feature	GitBase	Real Git
Staging	Copies files to `staging/` directory	Stores file hashes in a binary index file
Commits	Full file copies in a UUID-named directory	Content-addressable objects (blobs) deduplicated by SHA hash
Commit ID	Random UUID (v4)	SHA-1/SHA-256 hash of commit content (deterministic)
History	Flat list of directories (no parent linking)	Linked chain — each commit points to its parent
Branches	Not implemented	Named pointers to commits
Diffs	Not implemented (stores full copies)	Computed on-the-fly from stored objects
Remote	AWS S3 (file storage)	Git protocol (smart HTTP/SSH)
Merge	Not implemented	Three-way merge algorithm

What the Empty Files Are Meant to Become

models/userModel.js — Would look like:

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const userSchema = new mongoose.Schema({
  username:  { type: String, required: true, unique: true },
  email:     { type: String, required: true, unique: true },
  password:  { type: String, required: true },
  avatar:    { type: String, default: "" },
  repos:     [{ type: mongoose.Schema.Types.ObjectId, ref: "Repo" }],
  createdAt: { type: Date, default: Date.now },
});

userSchema.pre("save", async function(next) {
  if (!this.isModified("password")) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.comparePassword = async function(candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};

module.exports = mongoose.model("User", userSchema);
```

models/repoModel.js — Would look like:

```
const mongoose = require("mongoose");

const repoSchema = new mongoose.Schema({
  name:        { type: String, required: true },
  description: { type: String, default: "" },
  owner:       { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  visibility:  { type: String, enum: ["public", "private"], default: "public" },
  s3Key:       { type: String },
  stars:       [{ type: mongoose.Schema.Types.ObjectId, ref: "User" }],
  forks:       { type: Number, default: 0 },
  createdAt:   { type: Date, default: Date.now },
  updatedAt:   { type: Date, default: Date.now },
});

module.exports = mongoose.model("Repo", repoSchema);
```

models/issueModel.js — Would look like:

```
const mongoose = require("mongoose");

const issueSchema = new mongoose.Schema({
  title:    { type: String, required: true },
  body:     { type: String, default: "" },
  status:   { type: String, enum: ["open", "closed"], default: "open" },
  repo:     { type: mongoose.Schema.Types.ObjectId, ref: "Repo", required: true },
  author:   { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  assignee: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  labels:   [{ type: String }],
  comments: [{
    author:    { type: mongoose.Schema.Types.ObjectId, ref: "User" },
    body:      String,
    createdAt: { type: Date, default: Date.now }
  }],
  createdAt: { type: Date, default: Date.now },
});

module.exports = mongoose.model("Issue", issueSchema);
```

middleware/authMiddleware.js — Would look like:

```
const jwt = require("jsonwebtoken");
const User = require("../models/userModel");

async function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ error: "No token provided" });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.userId).select("-password");
    if (!req.user) {
      return res.status(401).json({ error: "User not found" });
    }
    next();
  } catch (err) {
    return res.status(401).json({ error: "Invalid token" });
  }
}

module.exports = authMiddleware;
```

middleware/authorizeMiddleware.js — Would look like:

```
function authorizeMiddleware(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ error: "Not authenticated" });
    }
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: "Not authorized" });
    }
    next();
  };
}

module.exports = authorizeMiddleware;
```

Bugs & Issues Found

#	File	Issue	Severity
1	init.js (Line 5)	Uses `".gitBase"` (lowercase 'g') but all other files use `".GitBase"` (uppercase 'G'). On Linux/Mac (case-sensitive filesystems), `init` creates a different directory than `add` / `commit` / `push` / `pull` / `revert` look at	HIGH
2	index.js (Lines 73-74)	Both `bodyParser.json()` and `express.json()` are registered — they're redundant, remove one	LOW
3	index.js (Lines 88-100)	`let user = "test"` is shared across all connections. When a second user connects, it overwrites the first user's value. Should be per-socket or use a Map	MEDIUM
4	pull.js (Line 36)	`console.log("All commits pulled from S3!")` is inside the `for` loop — logs once per file instead of once at the end	LOW
5	revert.js (Lines 17-18)	Copies **all** files from the commit dir including `commit.json` into the working directory. The `commit.json` metadata file shouldn't be restored as a project file	MEDIUM
6	pull.js (Line 34)	Writes file to `path.join(repoPath, key)` which creates `.GitBase/commits/uuid/file`. But `key` already starts with `"commits/"`, so it could create a double-nested path	MEDIUM
7	.env	Contains hardcoded MongoDB credentials. The file is in `.gitignore`, but the local `.GitBase` commit history contains it as a tracked file	HIGH
8	commit.js	Staging area is never cleared after commit — the same files will be re-committed next time	MEDIUM
9	push.js	Pushes **all** commits every time (no tracking of what's already pushed). For large histories this wastes bandwidth	LOW

Security Analysis

Area	Current State	Risk
Authentication	None — no middleware implemented	Anyone can hit any endpoint
CORS	`origin: "*"` — allows all origins	Any website can make requests
MongoDB credentials	Hardcoded in `.env` with read access in `.GitBase/`	Credential exposure
AWS credentials	Expected from env/config (not hardcoded — good)	Depends on IAM policy
Input validation	None on CLI args or API bodies	Path traversal possible in `add <file>`
HTTPS	Not configured	Traffic is unencrypted
Rate limiting	Not implemented	Vulnerable to DoS

WARNING: The `add` command takes a raw file path from the user and passes it directly to `fs.copyFile()`. A malicious path like `../../etc/passwd` could stage sensitive files. Always validate and sanitize file paths.

How to Run the Project

CLI Commands (Version Control)

```
cd S:\GitBase\backend

# Initialize a new repo
node index.js init

# Stage a file
node index.js add hello.txt

# Commit staged files
node index.js commit "My first commit"

# Push to AWS S3
node index.js push

# Pull from AWS S3
node index.js pull

# Revert to a previous commit
node index.js revert 03f5f82f-3128-4f98-8ed6-75f4a5caf8bd
```

Web Server

```
# Start the Express + Socket.IO server
node index.js start

# Or with auto-restart on file changes (dev mode)
npx nodemon index.js start
```

The server will:

1. Connect to MongoDB Atlas
2. Start listening on port 3000
3. Accept WebSocket connections
4. Serve GET / → "Welcome!"

Architecture Summary — What's Built vs What's Planned

IMPLEMENTED (Working):

- CLI Commands: init, add, commit, push, pull, revert
- Express Server Shell
- Socket.IO Connection handling
- MongoDB Connection

PLACEHOLDER (TODO):

- Auth Middleware (JWT)
- Authorization Middleware
- User Model
- Repo Model
- Issue Model
- API Routes

The project has a solid CLI-based version control core and a server skeleton ready for the GitHub-like web features (user accounts, repositories, issues, pull requests, etc.).